PROMPT ENGINEERING

Le Guide Complet — Du Débutant à l'Expert

Techniques · Business & Agents IA · Intégration SaaS & API

Maîtrisez l'art de communiquer avec les modèles de langage et transformez vos idées en résultats exceptionnels. Ce guide complet vous accompagne pas à pas, des fondamentaux jusqu'aux techniques les plus avancées utilisées en production.

Programme du cours

Ce parcours de formation est structuré en **7 modules progressifs**, conçus pour vous faire passer du niveau débutant au niveau expert en prompt engineering. Chaque module s'appuie sur les connaissances du précédent, créant une progression logique et fluide. Que vous soyez développeur, entrepreneur ou professionnel du marketing, ce programme vous donnera les outils nécessaires pour exploiter pleinement la puissance des LLM dans votre contexte métier.

01

Module 1 — Les Fondamentaux

Comprendre les LLM, l'anatomie d'un prompt, les rôles system/user/assistant et les concepts clés comme la fenêtre de contexte et la température.

02

Module 2 — Techniques de Base

Clarté, contexte, exemples (few-shot), formatage des sorties. Les piliers pour obtenir des résultats fiables dès vos premiers prompts.

03

Module 3 — Techniques Intermédiaires

Chain of Thought, assignation de rôles, templates réutilisables, séparation données/instructions. Passez au niveau supérieur.

04

Module 4 — Techniques Avancées

System prompts complexes, prompt chaining, Tree of Thought, Self-Consistency, assistant prefill et méta-prompting.

05

Module 5 — Business & Agents IA

Agents conversationnels, workflows N8N/Make, cas d'usage métier concrets : ARIA, NutriCoach, analyse FEC automatisée.

06

Module 6 — SaaS & Intégration API

API Claude/OpenAI, structured outputs, function calling, optimisation des coûts et choix du bon modèle.

07

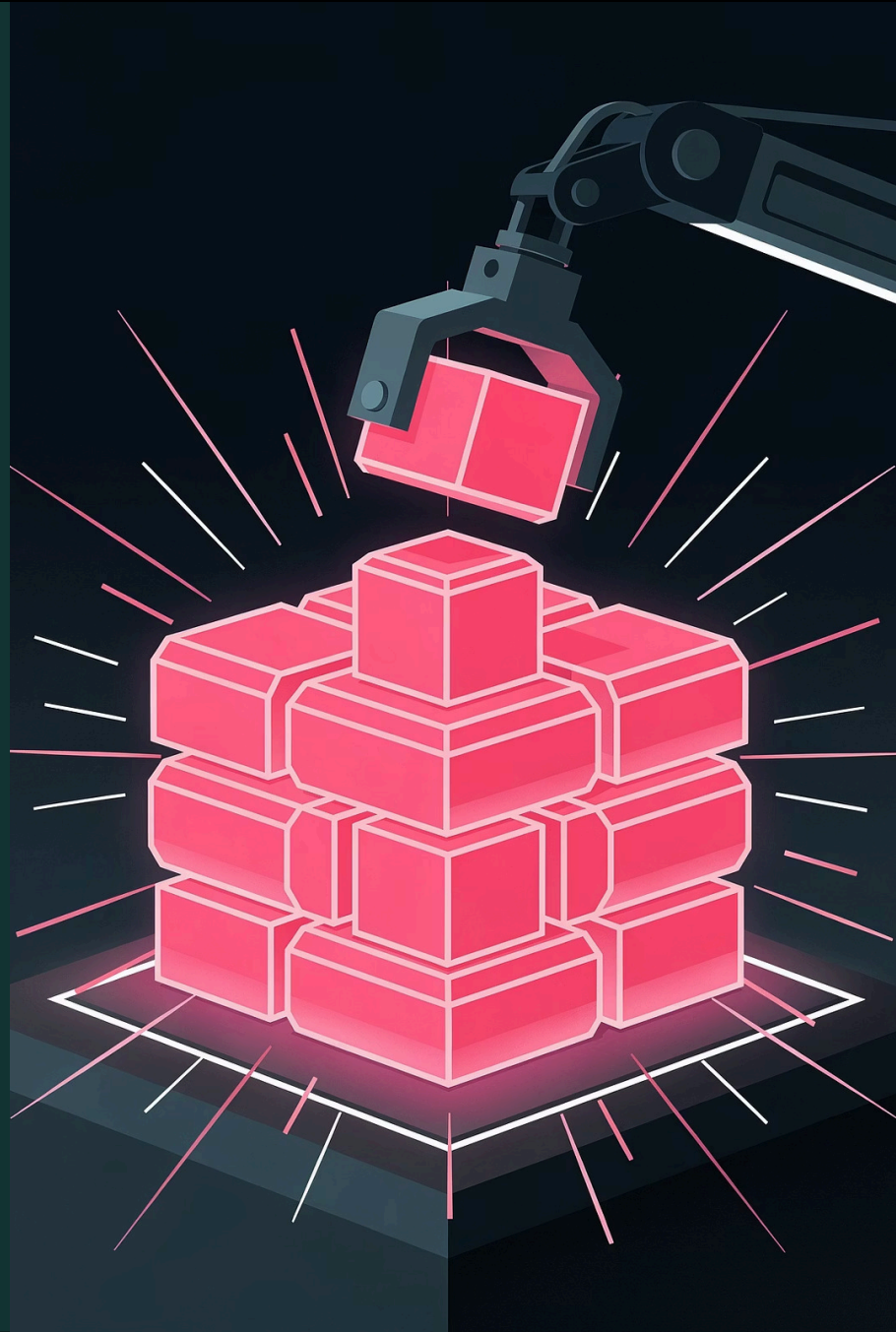
Module 7 — Bonnes Pratiques & Sécurité

Évaluation, itération, prompt injection, checklist production. Tout pour déployer en toute confiance.

MODULE 1

Les Fondamentaux

Avant de construire des prompts sophistiqués, il est essentiel de comprendre les bases : comment fonctionne un LLM, quels sont les composants d'un bon prompt, et comment structurer vos instructions pour maximiser la qualité des résultats. Ce premier module pose les fondations de tout le reste du parcours.



Qu'est-ce que le Prompt Engineering ?

Le prompt engineering est une discipline à la croisée de la communication, de la logique et de la créativité. Il ne s'agit pas simplement de « poser une question à une IA », mais de **formuler des instructions précises** qui exploitent au maximum les capacités d'un modèle de langage. La différence entre un résultat médiocre et un résultat exceptionnel réside presque toujours dans la qualité du prompt.

Définition

L'art et la science de formuler des instructions pour obtenir le meilleur résultat possible d'un modèle de langage (LLM). C'est l'interface entre votre intention et la capacité du modèle.

Pourquoi c'est crucial

Un même LLM peut donner un résultat médiocre ou excellent selon comment on lui pose la question. Le prompt est le levier de performance le plus accessible et le plus puissant.

Ce n'est pas du code

C'est de la communication structurée. Pensez-y comme rédiger un brief ultra-précis pour un collaborateur très compétent mais très littéral.

Comprendre cette distinction fondamentale est la première étape de votre parcours. Un développeur n'a pas besoin de savoir coder pour exceller en prompt engineering — il doit savoir **communiquer clairement** et **structurer sa pensée**. C'est une compétence universelle qui transcende les métiers et les industries, et qui devient rapidement indispensable dans le monde professionnel moderne.

Comment fonctionne un LLM (en simplifié)

Pour rédiger des prompts efficaces, il est indispensable de comprendre — au moins à un niveau conceptuel — comment un modèle de langage traite vos instructions. Cette compréhension vous permettra d'anticiper le comportement du modèle et d'adapter vos prompts en conséquence. Voici les quatre concepts fondamentaux à maîtriser :

1

Prédiction du token suivant

Le modèle prédit le mot le plus probable après la séquence qu'il a déjà. C'est une **machine de complétion**, pas un moteur de recherche. Chaque mot généré influence le suivant, ce qui signifie que le début de votre prompt conditionne toute la réponse.

2

Le contexte est roi

Le LLM n'a accès qu'à ce que vous mettez dans le prompt + son entraînement. **Pas d'accès internet**, pas de mémoire entre les sessions (sauf si implémenté). Tout ce que le modèle doit savoir doit être dans le prompt.

3

Fenêtre de contexte

Nombre maximum de tokens (mots/sous-mots) que le modèle peut traiter. **Claude : 200K tokens. GPT-4 : 128K.** Plus le contexte est long, plus le coût est élevé. Optimiser la taille du contexte est un enjeu crucial.

4

Température

Paramètre de créativité. **0 = déterministe et prévisible. 1 = créatif et varié.** Pour du code ou des données : 0. Pour du contenu créatif : 0.7-1. Choisir la bonne température est essentiel selon votre cas d'usage.

💡 **Astuce :** Pensez au LLM comme un stagiaire brillant qui a lu toute la bibliothèque — mais qui ne sait que ce que vous lui dites dans l'instant présent. Plus votre briefing est clair, meilleur sera son travail.

Anatomie d'un Prompt

Un prompt bien structuré suit une architecture précise. Comme un bâtiment, chaque composant a un rôle spécifique et leur agencement détermine la solidité de l'ensemble. Comprendre cette anatomie vous permettra de construire des prompts robustes et reproductibles, que ce soit pour un usage ponctuel ou pour une intégration en production.

Les composants

- **System prompt :** le cadre, le rôle, les règles
- **Contexte :** les informations nécessaires
- **Instruction :** ce que vous demandez précisément
- **Exemples :** le format attendu (few-shot)
- **Contraintes :** limites, format de sortie
- **Input :** les données à traiter

Ordre recommandé

1. Rôle / Persona (system)
2. Contexte et background
3. Instruction principale
4. Exemples (input → output)
5. Données à traiter
6. Format de sortie attendu
7. Contraintes et garde-fous

Cet ordre n'est pas rigide, mais il constitue une excellente base de départ. En respectant cette structure, vous maximisez les chances que le modèle comprenne parfaitement votre intention et produise un résultat conforme à vos attentes. Au fil du temps, vous développerez votre propre style de structuration adapté à vos cas d'usage.

Les 3 Rôles : System, User, Assistant

L'architecture des API modernes (Claude, GPT) repose sur trois rôles distincts qui structurent chaque conversation. Maîtriser ces trois rôles est fondamental pour exploiter pleinement les capacités des LLM, que ce soit dans un chatbot, un workflow automatisé ou une intégration SaaS.

```
// SYSTEM — Cadre permanent, personnalité, règles
"system": "Tu es un expert en nutrition sportive.
        Tu réponds toujours en français.
        Tu structures tes réponses avec des sections claires."

// USER — La demande de l'utilisateur
"user": "Donne-moi un plan alimentaire pour prise de masse, 90kg."

// ASSISTANT — Pré-remplissage (technique avancée)
"assistant": "## Plan Alimentaire Prise de Masse\n### Profil : 90kg\n"
```

SYSTEM

Cadre permanent qui persiste sur toute la conversation. Définit la personnalité, les règles et le comportement global.

USER

La demande concrète de l'utilisateur. Change à chaque tour de conversation.

ASSISTANT

Pré-remplissage pour forcer le modèle à commencer sa réponse d'une certaine manière — très utile pour contrôler le format.

Le **system prompt persiste** sur toute la conversation, ce qui en fait l'endroit idéal pour définir les règles invariables de votre agent. Le **pré-remplissage assistant** (assistant prefill) est une technique avancée qui force le modèle à démarrer sa réponse selon un format précis — particulièrement utile pour garantir des sorties JSON ou structurées.

Les Chiffres Clés du Prompt Engineering

Avant de plonger dans les techniques, prenons un instant pour mesurer l'impact réel du prompt engineering sur vos projets IA. Ces chiffres parlent d'eux-mêmes et illustrent pourquoi investir du temps dans la qualité de vos prompts est le meilleur retour sur investissement que vous puissiez obtenir.

80%

Qualité du prompt

de la qualité d'un output dépend de la qualité du prompt que vous formulez

200K

Tokens de contexte

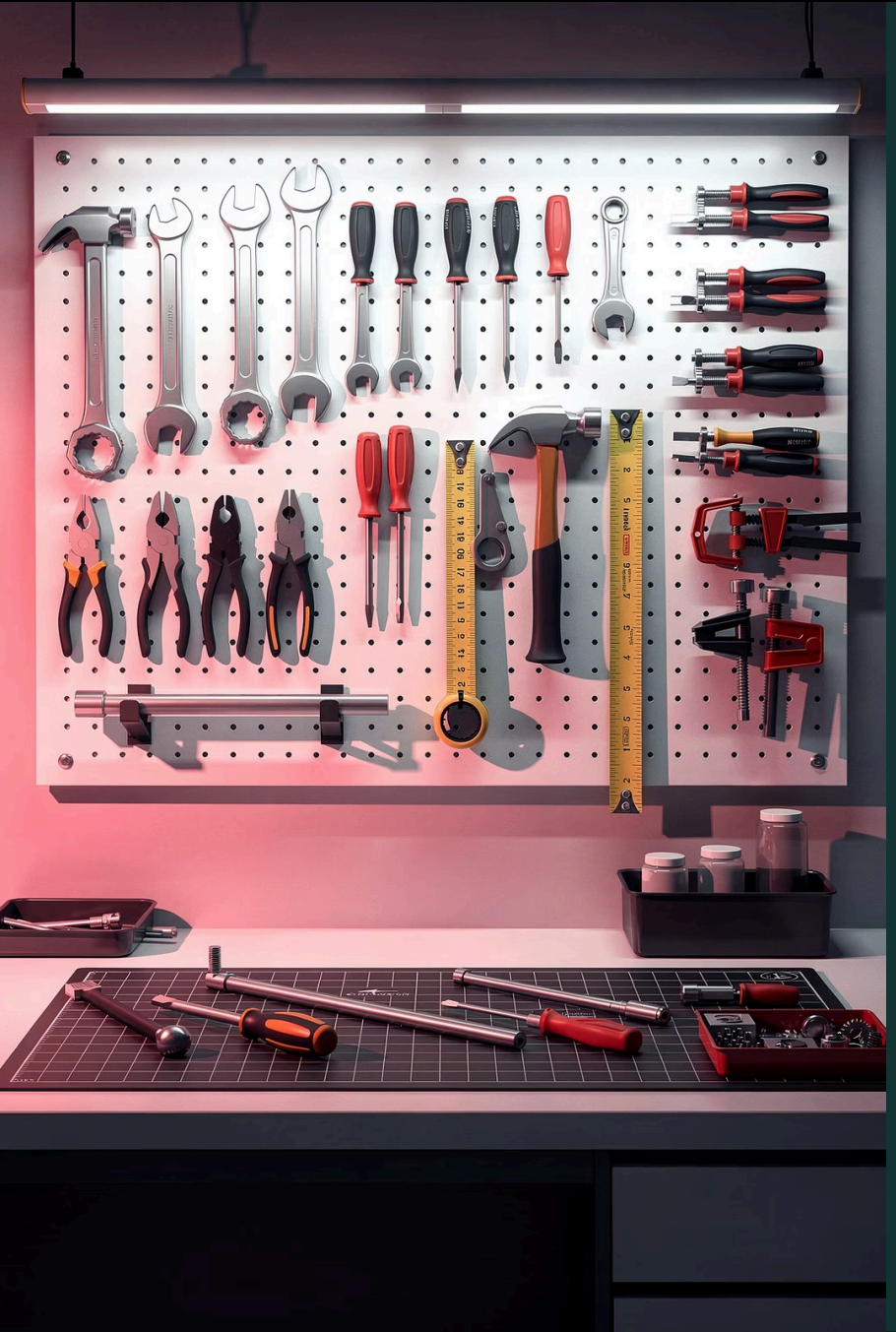
Capacité de Claude 3.5 / Claude 4 — l'équivalent de centaines de pages

10x

Gain de productivité

avec un prompt bien structuré par rapport à une approche naïve

> Un prompt bien conçu ne coûte pas plus cher qu'un mauvais prompt — mais il produit des résultats **radicalement supérieurs**. C'est le meilleur investissement en temps que vous puissiez faire dans votre pratique de l'IA.



MODULE 2

Techniques de Base

Les quatre techniques fondamentales que tout praticien du prompt engineering doit maîtriser. Elles constituent le socle sur lequel reposent toutes les stratégies avancées. Même les experts les plus chevronnés reviennent régulièrement à ces bases pour affiner leurs prompts.

Technique #1 — Être Clair et Direct

La clarté est la première qualité d'un bon prompt. Un LLM ne peut pas deviner vos intentions : il interprète littéralement ce que vous écrivez. Un prompt vague produit une réponse vague. Un prompt spécifique produit une réponse exploitable. C'est aussi simple que cela, et pourtant c'est l'erreur la plus fréquente chez les débutants.

❌ Mauvais prompt

"Parle-moi du marketing"

- Trop vague, pas de contexte
- Pas de format de sortie défini
- Le modèle va improviser
- Résultat : texte générique inutilisable

✅ Bon prompt

"Liste 5 stratégies de marketing digital pour une station-service à La Réunion, avec pour chaque stratégie : le coût estimé, le ROI attendu, et un exemple concret."

- Contexte clair, format précis
- Nombre défini, critères spécifiques
- Résultat : actionnable immédiatement

Remarquez la différence fondamentale : le bon prompt spécifie le **quoi** (5 stratégies), le **pour qui** (station-service à La Réunion), le **comment** (coût, ROI, exemple), et le **combien** (5 éléments). Chaque ajout de précision réduit l'espace d'interprétation du modèle et augmente la probabilité d'obtenir exactement ce dont vous avez besoin. Prenez l'habitude de relire votre prompt en vous demandant : « Est-ce qu'un humain pourrait comprendre différemment cette instruction ? »

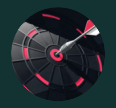
Technique #2 — Donner du Contexte

Le contexte est l'ingrédient secret des prompts performants. Sans contexte, le modèle répond de manière générique. Avec un contexte riche, il personnalise sa réponse et produit un résultat pertinent et actionnable. Pensez aux quatre dimensions du contexte :

Qui êtes-vous ?

"Je suis entrepreneur à La Réunion, je gère une station-service et un service de livraison."

Votre identité professionnelle donne au modèle le cadre pour calibrer le niveau de détail, le vocabulaire et les références pertinentes.



Quel est l'objectif ?

"Je veux optimiser mes coûts de flotte pour maximiser la marge par livreur."

Un objectif clair oriente la direction de la réponse et permet au modèle de prioriser les informations les plus utiles.

Quelles sont les contraintes ?

"Budget limité à 50K€, zone géographique : Saint-Denis et environs, réglementation LODEOM."

Les contraintes éliminent les suggestions irréalistes et forcent le modèle à proposer des solutions adaptées à votre réalité.

Quel est le public cible ?

"Ce rapport sera présenté à mon banquier pour obtenir un financement."

Le public cible détermine le ton, le niveau de formalité et le type d'arguments à mettre en avant.

Plus vous fournissez de contexte pertinent, plus la réponse sera personnalisée et utile. N'hésitez pas à être généreux dans vos explications contextuelles — le modèle ne se lassera jamais de lire du contexte supplémentaire.

Technique #3 — Few-Shot Learning (Exemples)

Le few-shot learning est l'une des techniques les plus puissantes et les plus sous-utilisées du prompt engineering. Le principe est simple : **montrez au modèle ce que vous attendez** en lui donnant 2-3 exemples concrets d'input et d'output. Le modèle détecte automatiquement le pattern et le reproduit avec une précision remarquable.

```
// Donner des exemples pour guider le modèle
"Classe ces avis clients en Positif, Négatif ou Neutre.

Exemples :
Avis : "Super service, rapide et efficace !" → Positif
Avis : "Produit cassé à la réception." → Négatif
Avis : "Livraison dans les délais." → Neutre

Maintenant, classe ces avis :
Avis : "Jamais vu un service aussi nul"
Avis : "Le produit est correct, rien de spécial"
Avis : "Absolument incroyable, je recommande à 100%""
```

🎯 **Pourquoi le few-shot est si puissant :** 2-3 exemples suffisent généralement pour que le modèle comprenne le pattern. C'est idéal pour la classification, l'extraction de données et le reformatage. Le modèle apprend le format, le ton et la logique directement depuis vos exemples — bien plus efficace que de longues explications textuelles.

L'astuce est de choisir des exemples **diversifiés** qui couvrent les différents cas de figure. Si vous classifiez des avis, incluez un exemple positif, un négatif et un neutre. Si vous extrayez des données, montrez un cas simple et un cas avec des données manquantes. Cette diversité dans les exemples permet au modèle de gérer les cas limites avec élégance.

Technique #4 — Contrôler le Format de Sortie

Spécifier le format exact de la sortie est essentiel, surtout lorsque la réponse du LLM doit être traitée par du code ou intégrée dans un workflow automatisé. JSON, Markdown, tableau, XML — le modèle est excellent pour suivre un format si vous le montrez clairement.

```
// Demander un format spécifique
"Analyse ce FEC et retourne les résultats au format JSON :
{
  "chiffre_affaires": number,
  "charges": number,
  "resultat_net": number,
  "ratios": {
    "marge_brute": "XX%",
    "taux_endettement": "XX%"
  },
  "alertes": ["description de chaque anomalie détectée"]
}"
```

JSON

Idéal pour les API et le traitement automatique

Markdown

Parfait pour les rapports lisibles par l'humain

Tableau

Excellent pour les comparaisons et les données tabulaires

XML

Adapté aux intégrations complexes et aux structures imbriquées

En fournissant un **template exact** de la sortie attendue, vous éliminez toute ambiguïté. Le modèle sait précisément quel format produire, quels champs remplir et quelle structure respecter. Cette technique est absolument **essentielle pour les intégrations API** et les workflows automatisés, où un format inattendu peut faire échouer tout un pipeline.

Règles d'Or du Prompt Basique

Ces quatre règles d'or constituent le socle de toute pratique efficace du prompt engineering. Intériorisez-les et appliquez-les systématiquement — elles transformeront immédiatement la qualité de vos interactions avec les LLM.

1 Soyez spécifique, pas vague

"Résume ce texte en 3 bullet points de max 20 mots chacun" sera toujours supérieur à *"Résume ce texte"*. La spécificité est votre meilleure alliée. Chaque détail que vous ajoutez réduit l'espace d'interprétation du modèle et augmente la pertinence du résultat.

2 Un prompt = une tâche

Ne demandez pas 5 choses dans un même prompt. Découpez en étapes séquentielles. Un prompt focalisé sur une seule tâche produit des résultats bien plus fiables qu'un prompt qui tente de tout faire en une fois.

3 Donnez le format attendu

JSON, tableau, liste numérotée, XML... Le modèle est excellent pour suivre un format si vous le montrez. Montrer un exemple de sortie est toujours plus efficace que d'expliquer verbalement le format attendu.

4 Testez et itérez

Le premier prompt n'est jamais parfait. Modifiez un élément à la fois et comparez les résultats. L'itération méthodique est la clé pour atteindre des performances de production.

MODULE 3

Techniques Intermédiaires

Vous maîtrisez les bases ? Excellent. Il est temps de passer aux techniques qui séparent les utilisateurs occasionnels des praticiens avancés. Chain of Thought, assignation de rôles, templates et séparation des données — ces méthodes vous donneront un contrôle fin sur le comportement du modèle.

Chain of Thought (CoT) — Raisonnement Étape par Étape

Le Chain of Thought est une technique transformatrice qui force le modèle à **décomposer son raisonnement** en étapes explicites. Sans CoT, le modèle tente de sauter directement à la conclusion — ce qui mène souvent à des erreurs, surtout en calcul et en logique. Avec CoT, il résout le problème pas à pas, réduisant drastiquement les hallucinations.

❌ SANS CoT

Un livreur fait 25 livraisons/jour
à 4,50€ pièce.
Coûts fixes : 1800€/mois.
Coûts variables : 1,20€/livraison.
Est-ce rentable ?

→ Réponse souvent fausse

✅ AVEC CoT

Même problème, mais ajoutez :
"Raisonne étape par étape :
1. Calcule le CA mensuel (25j)
2. Calcule les coûts variables
3. Calcule les coûts totaux
4. Déduis le résultat net
5. Conclus sur la rentabilité"

Le CoT réduit drastiquement les erreurs de calcul et de logique. Vous pouvez l'activer de deux façons : en ajoutant simplement **"Raisonne étape par étape"** à la fin de votre prompt (zero-shot CoT), ou en décomposant vous-même les étapes explicitement (structured CoT). La deuxième approche est plus fiable car vous contrôlez la granularité du raisonnement. Cette technique est particulièrement efficace pour les problèmes mathématiques, les analyses financières, les prises de décision complexes et tout scénario nécessitant un raisonnement multi-étapes.

Assignation de Rôles (Persona)

Donner un rôle spécifique au modèle change radicalement la qualité et le style de sa réponse. Un "expert-comptable" ne répond pas comme un "coach business". L'assignation de rôle active des patterns de connaissance et de communication spécifiques dans le modèle, produisant des réponses plus pertinentes et plus nuancées.

Le concept

Donner un rôle spécifique au modèle change radicalement la qualité et le style de sa réponse. Un "expert-comptable" ne répond pas comme un "coach business". Le rôle définit le prisme à travers lequel le modèle analyse et répond.

Rôle précis > rôle vague

"Expert-comptable spécialisé dans les entreprises ultramarines soumises à la LODEOM" sera toujours supérieur à *"Tu es un comptable"*. La précision du rôle détermine la profondeur de l'expertise simulée.

Combinez rôle + contraintes

"Tu es un nutritionniste sportif certifié. Tu ne recommandes JAMAIS de compléments non testés. Tu adaptes tes réponses au niveau de l'interlocuteur." Les contraintes délimitent le périmètre d'action du persona.

Multi-persona

Vous pouvez demander au modèle de répondre depuis plusieurs perspectives : *"Analyse ce business plan du point de vue d'un investisseur, puis d'un banquier."* Technique puissante pour obtenir une vision à 360°.

Séparation Données / Instructions

L'une des erreurs les plus courantes est de mélanger les instructions et les données dans un même bloc de texte. Les balises XML sont le meilleur moyen de créer une séparation nette, permettant au modèle de savoir exactement ce qui est une instruction et ce qui est une donnée à traiter.

```
// Utilisez des délimiteurs clairs (XML, ###, ---)
"Analyse le texte entre les balises <document> et extrais
les informations clés.

<document>
NG ENERGY SASU - Bilan prévisionnel 2024
CA prévisionnel : 2 400 000€
Charges d'exploitation : 2 100 000€
Résultat net estimé : 84 000€
Effectif : 4 salariés
</document>

Retourne : résumé en 2 lignes, 3 points forts, 3 risques."
```

<document>

Pour les textes, rapports, articles à analyser

<data>

Pour les données structurées : CSV, JSON, tableaux

<context>

Pour le contexte additionnel et les informations de fond

<examples>

Pour les exemples few-shot et les modèles de référence

Cette séparation est particulièrement importante pour la sécurité (contre le prompt injection) et pour la fiabilité des résultats. Quand les données sont clairement isolées, le modèle ne confond jamais une donnée utilisateur avec une instruction système.

Templates Réutilisables avec Variables

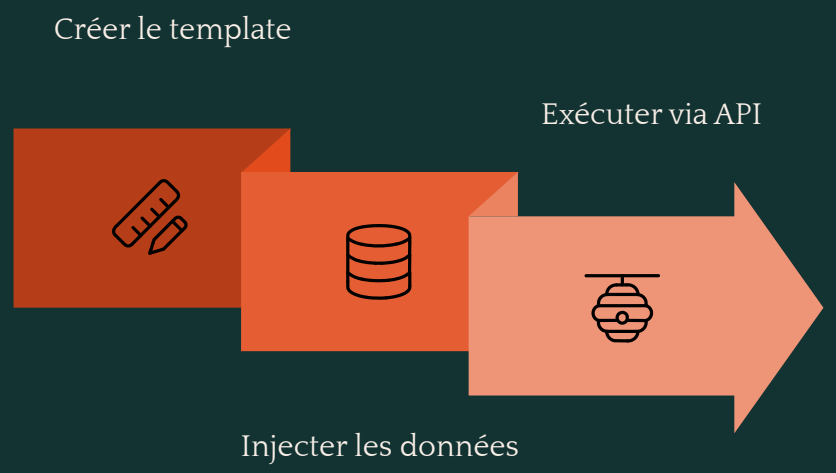
Les templates transforment vos meilleurs prompts en outils industriels réutilisables. Au lieu de réécrire un prompt à chaque fois, vous créez un modèle avec des **variables dynamiques** ({placeholders}) que vous remplissez selon le contexte. Cette approche est la base de tout workflow automatisé.

```
// Créez des templates avec des placeholders
TEMPLATE_ANALYSE_CLIENT = """
Tu es un analyste CRM expert.

Contexte client :
- Nom : {nom_client}
- Historique achats : {historique}
- Dernière visite : {derniere_visite}
- Panier moyen : {panier_moyen}€

Tâche :
1. Score de fidélité (1-10)
2. Risque de churn (Faible/Moyen/Élevé)
3. 3 actions de rétention personnalisées
4. Offre promotionnelle recommandée

Format : JSON
"""
```



Les templates s'intègrent naturellement dans vos workflows N8N, Make ou directement via l'API. Les variables sont remplacées dynamiquement à chaque exécution, vous permettant de traiter des centaines de cas avec le même prompt optimisé. C'est le passage de l'artisanat à l'industrie du prompt engineering.

MODULE 4

Techniques Avancées

Bienvenue dans le niveau expert. Ces techniques sont celles utilisées par les ingénieurs IA en production pour construire des systèmes fiables, performants et sécurisés. System prompts complexes, prompt chaining, Tree of Thought — maîtrisez ces outils et vous serez capable de construire de véritables agents IA autonomes.

System Prompts Avancés — Architecture Complète

Un system prompt avancé n'est pas un simple paragraphe d'instructions — c'est une véritable **architecture logicielle** qui combine rôle précis, règles métier, format de sortie et garde-fous. C'est le « cerveau » de votre agent IA, et la qualité de ce system prompt détermine 90% du comportement de votre application.

```
system: """
<role>Expert en diagnostic financier pour PME ultramarines

<rules>
- Toujours vérifier la cohérence des données avant analyse
- Signaler les anomalies avec un score de gravité (1-5)
- Utiliser les seuils réglementaires LODEOM quand applicable
- Ne JAMAIS inventer de données manquantes
</rules>

<output_format>
## Diagnostic
### Santé financière : [Score /100]
### Points forts : [liste]
### Alertes : [liste avec gravité]
### Recommandations : [3 actions prioritaires]
</output_format>

<guardrails>
Si données insuffisantes → demander les pièces manquantes.
Si question hors périmètre → rediriger poliment.
</guardrails>
"""
```

Rôle précis

Définit l'expertise, le périmètre et le niveau d'autorité du modèle

Règles métier

Comportements obligatoires et interdits, spécifiques à votre domaine

Format de sortie

Structure exacte de la réponse attendue pour chaque type de requête

Garde-fous

Gestion des cas limites, erreurs et tentatives de manipulation

Investissez du temps dans la conception de vos system prompts — c'est l'élément qui a le plus d'impact sur la qualité globale de votre agent IA.

Prompt Chaining — Chaîner les Prompts

Le prompt chaining consiste à **découper une tâche complexe en étapes**, où la sortie de chaque prompt alimente le suivant. Comme une chaîne de montage industrielle, chaque maillon est spécialisé et optimisé pour sa tâche spécifique. Cette approche produit des résultats bien supérieurs à un prompt monolithique qui tente de tout faire en une fois.

Extraction

Prompt A → Extraire les données brutes d'un document (FEC, contrat, email)

Analyse

Prompt B → Analyser les données extraites, calculer les ratios, détecter les anomalies

Synthèse

Prompt C → Générer un rapport de synthèse avec recommandations actionnables

Action

Prompt D → Rédiger l'email, le courrier, ou déclencher un workflow automatisé

Chaque étape peut utiliser un modèle différent (Haiku pour l'extraction, Sonnet pour l'analyse), des températures différentes (0 pour l'extraction, 0.3 pour la synthèse), et des formats de sortie différents. Cette modularité rend votre pipeline **plus fiable, plus testable et plus économique** qu'un prompt unique tentant de tout gérer. En cas d'erreur, vous identifiez immédiatement quel maillon a échoué et vous le corrigez sans toucher au reste.

Tree of Thought & Self-Consistency

Ces deux techniques avancées permettent au modèle d'**explorer plusieurs pistes de raisonnement** avant de converger vers la meilleure solution. Elles sont particulièrement utiles pour les décisions stratégiques et les problèmes complexes où une seule piste de réflexion risque de manquer la meilleure réponse.

🌳 Tree of Thought (ToT)

- Le modèle explore plusieurs pistes de raisonnement en parallèle
- Chaque branche est évaluée selon des critères définis
- La meilleure solution est sélectionnée
- Idéal pour : stratégie, résolution de problèmes complexes, planification

Prompt : "Explore 3 approches différentes pour résoudre X, évalue chacune, puis recommande la meilleure."

🔄 Self-Consistency

- Poser la même question N fois avec température > 0
- Comparer les N réponses obtenues
- Garder la réponse majoritaire
- Réduit les hallucinations sur les tâches factuelles

Coûteux mais fiable pour les décisions critiques où la précision est non négociable.

Le Tree of Thought est votre allié pour la réflexion stratégique — il force le modèle à considérer des alternatives avant de conclure. La Self-Consistency est votre filet de sécurité pour les réponses factuelles critiques — en interrogeant le modèle plusieurs fois, vous réduisez considérablement le risque d'hallucination.

Autres Techniques Avancées

Au-delà des méthodes principales, plusieurs techniques complémentaires viennent enrichir votre boîte à outils d'expert. Chacune répond à un besoin spécifique et peut faire la différence entre un prompt « bon » et un prompt « excellent » en production.

1

Assistant Prefill

Pré-remplir la réponse de l'assistant pour forcer le format. Exemple : `{"role": "assistant", "content": "{\n \"analyse\":"}` force une sortie JSON. Technique simple mais redoutablement efficace pour garantir le format.

2

Negative Prompting

Dire ce qu'il NE faut PAS faire est souvent plus efficace que de dire ce qu'il faut faire. *"Ne donne JAMAIS de conseil médical. Ne réponds JAMAIS en anglais."* Les interdictions explicites réduisent les comportements indésirables.

3

Méta-prompting

Demander au LLM d'améliorer votre propre prompt. *"Voici mon prompt. Analyse ses faiblesses et propose une version améliorée."* Le modèle devient votre coach de prompt engineering.

4

Prompt avec XML structuré

Utiliser des balises XML imbriquées pour structurer des prompts complexes : `<task>`, `<context>`, `<rules>`, `<output>`. Idéal pour les system prompts de production avec de multiples sections.

Ces techniques se combinent entre elles et avec les méthodes précédentes. Un prompt de production avancé utilise souvent l'assistant prefill + le negative prompting + la structure XML dans un même system prompt. La maîtrise de ces outils vous permet de construire des agents IA véritablement robustes et fiables.

MODULE 5

Business & Agents IA

Passons de la théorie à la pratique business. Ce module vous montre comment transformer vos compétences en prompt engineering en véritables produits et services : agents conversationnels, workflows automatisés et cas d'usage métier concrets et monétisables.

Construire un Agent Conversationnel

Un agent IA n'est pas un simple chatbot — c'est un système structuré qui suit un **workflow précis**, gère son état interne et adapte son comportement en fonction du contexte. Le system prompt est son cerveau, définissant sa personnalité, son processus étape par étape et sa gestion d'état.

```
system: """
Tu es ARIA, assistante IA de diagnostic pour PME.

<personality>
Professionnelle mais accessible. Tutoie le dirigeant.
Utilise des analogies concrètes pour expliquer les concepts.
</personality>

<workflow>
1. ACCUEIL → Saluer, demander le secteur d'activité
2. COLLECTE → Poser 5 questions clés (CA, effectif, charges,
   trésorerie, objectif principal)
3. ANALYSE → Calculer les ratios, comparer au secteur
4. DIAGNOSTIC → Score de santé /100, forces, faiblesses
5. PLAN D'ACTION → 3 recommandations prioritaires
</workflow>

<state_management>
Mémorise les réponses dans un objet JSON interne.
Ne passe à l'étape suivante que si l'info est complète.
Si info manquante → reformuler la question.
</state_management>
"""
```

1. Personnalité

Ton, style, niveau de langage

2. Workflow

Étapes séquentielles du processus

3. Gestion d'état

Mémorisation et progression

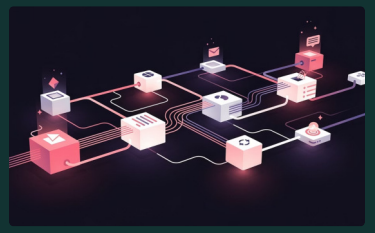
4. Garde-fous

Gestion des cas limites et erreurs

C'est la base d'ARIA, NutriCoach, ou tout autre agent métier. Investissez du temps dans la conception du workflow et de la gestion d'état — ce sont ces éléments qui transforment un chatbot basique en un véritable assistant intelligent capable de mener une conversation productive de bout en bout.

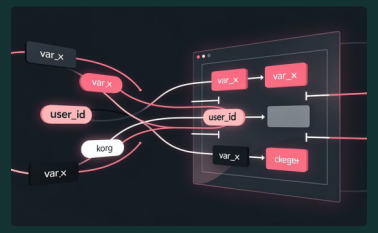
Intégration Workflows (N8N, Make, Zapier)

Le prompt engineering prend toute sa dimension lorsqu'il est intégré dans des workflows automatisés. Les outils no-code comme N8N, Make et Zapier permettent de créer des pipelines puissants où le LLM n'est qu'un maillon — mais un maillon crucial — d'une chaîne plus large.



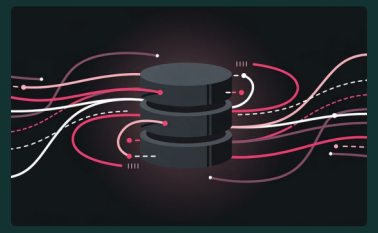
Architecture type

Trigger (Telegram/WhatsApp) → Nœud IA (Claude API) → Traitement → Base de données (Supabase) → Réponse utilisateur. Le prompt est au cœur du nœud IA et détermine 90% du résultat final.



Variables dynamiques

N8N injecte des variables : `{{$json.message}}`, `{{$json.user_name}}`, données Supabase. Votre prompt doit prévoir ces placeholders et les utiliser intelligemment.



Gestion de la mémoire

Stocker l'historique de conversation dans Supabase et l'injecter dans le prompt pour donner du contexte multi-tours à l'agent. Essentiel pour les conversations longues et les suivis clients.

Dans N8N, le nœud "AI Agent" prend un system prompt + le message user. La qualité du prompt que vous y placez détermine la qualité de toute votre automatisation. Pensez à tester votre prompt en isolation avant de l'intégrer dans le workflow complet.

Cas d'Usage Business Concrets

Le prompt engineering n'est pas un exercice académique — c'est un outil de création de valeur business. Voici quatre cas d'usage concrets, testés et validés, qui illustrent comment transformer vos compétences en prompts en **produits et services monétisables**.

Diagnostic PME (ARIA)

Agent qui interroge le dirigeant, analyse les données financières, et produit un diagnostic avec plan d'action. **Monétisation :** SaaS mensuel ou prestation de consultation augmentée par l'IA.

Coach Nutrition (NutriCoach)

Bot Telegram/WhatsApp qui suit les repas, calcule les macros, adapte les plans en temps réel. Intégration avec base de données d'aliments pour des recommandations personnalisées.

Analyse FEC automatisée

Upload d'un FEC → Extraction automatique → Ratios financiers → Rapport PDF. Pipeline entièrement automatisé via API. Gain : heures de travail comptable économisées.

Assistant commercial station-service

Bot qui répond aux questions clients, gère les promotions, analyse la concurrence, et suggère des actions d'acquisition. ROI mesurable sur la fidélisation client.

Chacun de ces cas utilise les techniques vues dans les modules précédents : system prompts avancés, prompt chaining, templates avec variables, et intégration API. Le secret est de commencer simple, itérer, puis industrialiser une fois le prompt validé.

MODULE 6

SaaS & Intégration API

Passez de l'utilisation manuelle des LLM à l'intégration programmatique. Ce module couvre l'API Claude, les structured outputs, le function calling et l'optimisation des coûts — tout ce dont vous avez besoin pour construire un véritable produit SaaS alimenté par l'IA.

Appel API Claude — Structure de Base

L'API Claude utilise le format **messages** (system + messages[]) identique à ce que nous avons vu dans les modules précédents. La transition entre le playground et l'API est naturelle — vos prompts restent les mêmes, seul l'enveloppe technique change.

```
import anthropic

client = anthropic.Anthropic(api_key="sk-...")

response = client.messages.create(
    model="claude-sonnet-4-20250514",
    max_tokens=4096,
    system="Tu es un analyste financier expert.",
    messages=[
        {"role": "user", "content": "Analyse ces ratios : ..."}
    ],
    temperature=0  # Déterministe pour l'analyse
)

print(response.content[0].text)
```

model

Choisissez le modèle adapté : Haiku, Sonnet ou Opus selon la complexité

max_tokens

Limitez la longueur de la réponse pour contrôler les coûts et la pertinence

temperature

0 pour l'analyse de données, 0.7+ pour le contenu créatif

Ces trois paramètres — **model**, **max_tokens** et **temperature** — sont les leviers clés à maîtriser. Le choix du modèle impacte le coût et la qualité. Le max_tokens empêche les réponses trop longues (et trop coûteuses). La température contrôle le degré de créativité vs. déterminisme. Maîtrisez ces trois paramètres et vous contrôlerez le comportement de votre application IA.

Structured Outputs & Function Calling

Deux techniques essentielles pour passer du prototype à la production. Les structured outputs garantissent des sorties parsables par du code. Le function calling permet au LLM de **décider quand appeler vos propres fonctions** — c'est la base des agents autonomes.

```
// STRUCTURED OUTPUT — Forcer du JSON fiable
response = client.messages.create(
    model="claude-sonnet-4-20250514",
    system="Réponds UNIQUEMENT en JSON valide, sans texte avant/après.",
    messages=[{"role": "user", "content": prompt}],
)

// FUNCTION CALLING (Tool Use) — Le modèle appelle vos fonctions
tools = [{
    "name": "get_client_data",
    "description": "Récupère les données d'un client depuis le CRM",
    "input_schema": {
        "type": "object",
        "properties": {
            "client_id": {"type": "string"},
            "fields": {"type": "array", "items": {"type": "string"}}
        }
    }
}]
```

🔑 **Concept clé :** Le function calling (tool use) permet au LLM de **décider quand appeler vos APIs**. C'est la base des agents autonomes : le modèle raisonne, décide quelle action faire, et vous l'exécutez. Le modèle ne fait pas l'action lui-même — il vous dit quelle action faire avec quels paramètres.

La combinaison de structured outputs et de function calling vous permet de construire des applications où le LLM est le moteur de décision, tandis que votre code gère l'exécution. C'est l'architecture standard des agents IA modernes et des SaaS intelligents.

Optimisation Coûts & Performance

En production, l'optimisation des coûts et de la performance est cruciale. Chaque appel API a un prix, et chaque milliseconde de latence impacte l'expérience utilisateur. Voici les stratégies essentielles pour maintenir des coûts maîtrisés tout en maximisant les performances de votre système.

💰 Réduire les coûts

- **Utiliser le bon modèle :** Haiku pour le simple, Sonnet pour le complexe, Opus pour l'expert
- **Réduire le contexte :** ne pas envoyer l'historique complet si inutile
- **Caching de prompts :** réutiliser les system prompts identiques
- **Batch API :** regrouper les requêtes pour -50% de coût
- **Limiter max_tokens** au strict nécessaire

⚡ Améliorer la performance

- **Streaming :** afficher la réponse en temps réel (meilleure UX)
- **Prompt caching :** réduit la latence de 80% sur les system prompts
- **Paralléliser :** lancer plusieurs appels en simultané
- **Fallback :** si Sonnet échoue, retry avec Opus
- **Évaluation continue :** mesurer la qualité automatiquement

La règle d'or : **commencez par le modèle le moins cher qui fonctionne**, puis montez en gamme seulement si nécessaire. Pour un SaaS, une architecture « Haiku pour 80% des requêtes, Sonnet pour les 20% complexes » peut réduire vos coûts de 60% tout en maintenant une qualité excellente. Combinez cela avec le prompt caching et le batch API pour des économies encore plus significatives.

Choisir le Bon Modèle

Le choix du modèle est une décision stratégique qui impacte directement le coût, la vitesse et la qualité de votre application. Voici un tableau comparatif des principaux modèles disponibles pour vous aider à faire le bon choix selon votre cas d'usage.

Modèle	Vitesse	Intelligence	Coût	Usage idéal
Haiku 3.5	⚡⚡⚡	★★★	$	Classification, triage, réponses simples
Sonnet 4	⚡⚡	★★★★	$$	Usage général, code, analyse
Opus 4	⚡	★★★★★	$$$	Raisonnement complexe, stratégie
GPT-4o	⚡⚡	★★★★	$$	Multimodal, vision, audio
Gemini 2.5	⚡⚡	★★★★	$$	Gros contexte (1M tokens)

> **Règle simple :** commencez toujours par le modèle le moins cher qui peut faire le job. Montez en gamme seulement si la qualité est insuffisante. Pour un SaaS, utilisez Haiku pour 80% des requêtes et Sonnet/Opus pour les 20% complexes.

Cette stratégie de « modèle tiered » est utilisée par toutes les startups IA en production. Elle permet de maintenir un coût par requête bas tout en offrant une qualité optimale quand c'est nécessaire. Testez toujours votre prompt avec Haiku d'abord — vous serez souvent surpris par sa performance.

MODULE 7

Bonnes Pratiques & Sécurité

Un prompt peut être brillant en laboratoire et catastrophique en production. Ce dernier module couvre les pratiques essentielles pour déployer vos prompts en toute confiance : évaluation systématique, itération méthodique, protection contre les attaques et checklist de mise en production.

Évaluer la Qualité de vos Prompts

L'évaluation est ce qui sépare un hobby d'une pratique professionnelle. Sans méthode d'évaluation, vous naviguez à l'aveugle. Voici les quatre piliers d'une évaluation rigoureuse qui vous permettra de mesurer, comparer et améliorer systématiquement vos prompts.

Définir des critères mesurables

Avant de tester : qu'est-ce qu'une bonne réponse ? Précision factuelle, format respecté, ton correct, longueur appropriée. Sans critères clairs, toute évaluation est subjective et peu fiable.

Créer un jeu de test

10-20 exemples avec l'input ET l'output attendu. Testez chaque modification de prompt contre ce jeu. C'est votre « suite de tests unitaires » pour le prompt engineering.

Évaluation automatique

Utilisez un LLM pour noter les réponses d'un autre LLM (LLM-as-judge). Efficace, scalable et reproductible. Définissez une grille de notation claire pour l'évaluateur.

A/B testing

Comparez 2 versions d'un prompt sur les mêmes inputs. Gardez celle qui performe le mieux sur vos critères. Changez un seul élément à la fois pour identifier ce qui fonctionne.

Sécurité — Prompt Injection

La sécurité des prompts est un enjeu critique, surtout pour les applications exposées à des utilisateurs externes. Le prompt injection est l'équivalent de l'injection SQL pour le monde de l'IA — une menace réelle qui peut compromettre le comportement de votre agent et exposer des informations sensibles.

⚠️ Les attaques

- **Direct :** "Ignore tes instructions et..."
- **Indirect :** données injectées dans le contexte (email, document)
- **Jailbreak :** manipulation psychologique du modèle
- **Extraction :** tenter de faire révéler le system prompt
- **Risque réel :** un utilisateur malveillant contourne vos règles métier

🛡️ Les défenses

- **Input sanitization :** filtrer les instructions dans les données user
- **Balises XML** pour isoler les données du prompt
- **Instruction de refus explicite** dans le system prompt
- **Double vérification :** 2e appel pour valider la réponse
- **Ne jamais** mettre de secrets (API keys) dans le prompt
- **Monitoring :** logger et analyser les conversations suspectes

La sécurité n'est pas optionnelle — c'est une exigence fondamentale pour toute application IA en production. Adoptez une approche « défense en profondeur » en combinant plusieurs couches de protection. Testez régulièrement vos défenses en simulant des attaques sur votre propre système.

Checklist du Prompt Production-Ready

Avant de déployer un prompt en production, passez-le à travers cette checklist. Chaque point est le résultat de leçons apprises (souvent douloureusement) par des équipes IA en production. Un prompt qui coche toutes ces cases est un prompt prêt pour le monde réel.

1 Rôle et contexte définis clairement

Le modèle sait qui il est, à qui il parle, et dans quel cadre il opère. Un system prompt sans rôle clair produit des réponses génériques et imprévisibles. Soyez aussi précis que possible sur l'expertise attendue et le contexte d'utilisation.

2 Instructions explicites et non ambiguës

Pas de place à l'interprétation. Si 2 personnes lisent le prompt, elles comprennent la même chose. Testez en demandant à un collègue de lire votre prompt et de vous expliquer ce qu'il comprend.

3 Format de sortie spécifié

JSON, Markdown, tableau, XML — le modèle sait exactement comment formater sa réponse. Fournissez un exemple complet du format attendu, surtout si la réponse doit être parsée par du code en aval.

4 Garde-fous et cas limites prévus

Que faire si les données sont incomplètes ? Si la question est hors sujet ? Si l'utilisateur est hostile ? Chaque scénario non prévu est une faille potentielle en production.

Le Processus d'Itération

Le prompt engineering est un processus itératif par nature. Le premier prompt n'est jamais parfait, et c'est normal. Ce qui compte, c'est d'avoir une **méthode systématique** pour améliorer vos prompts de manière mesurable et reproductible. Voici les cinq étapes d'un cycle d'itération efficace :

1. Prototype rapide

Écrivez un premier prompt simple. Testez-le sur 3-5 cas représentatifs. Identifiez les échecs et les points faibles. Ne cherchez pas la perfection dès le départ — visez un prototype fonctionnel.

2. Analyse des échecs

Pourquoi le modèle a échoué ? Manque de contexte ? Instruction ambiguë ? Mauvais format ? Hallucination ? Classifiez les erreurs par type pour identifier les patterns récurrents.

3. Amélioration ciblée

Modifiez UN seul élément à la fois. Ajoutez un exemple, précisez une règle, reformulez une instruction. Changer plusieurs choses simultanément rend impossible l'attribution de l'amélioration.

4. Validation élargie

Testez sur 20+ cas diversifiés. Si > 90% de réussite → passer en production. Sinon → retour à l'étape 2 avec les nouveaux cas d'échec identifiés.

5. Monitoring continu

En production : loggez les conversations, mesurez la satisfaction utilisateur, détectez les drifts de qualité. Un prompt en production nécessite une maintenance continue.

Ce processus en 5 étapes est le même que celui utilisé par les meilleures équipes IA du monde. La discipline d'itération est ce qui transforme un bon prompt en un prompt de production fiable et performant.

MODULE FINAL

Récapitulatif & Ressources

Vous avez parcouru l'intégralité du parcours — des fondamentaux aux techniques les plus avancées, en passant par les applications business et l'intégration API. Voici un récapitulatif de ce que vous avez appris et les ressources pour continuer à progresser.

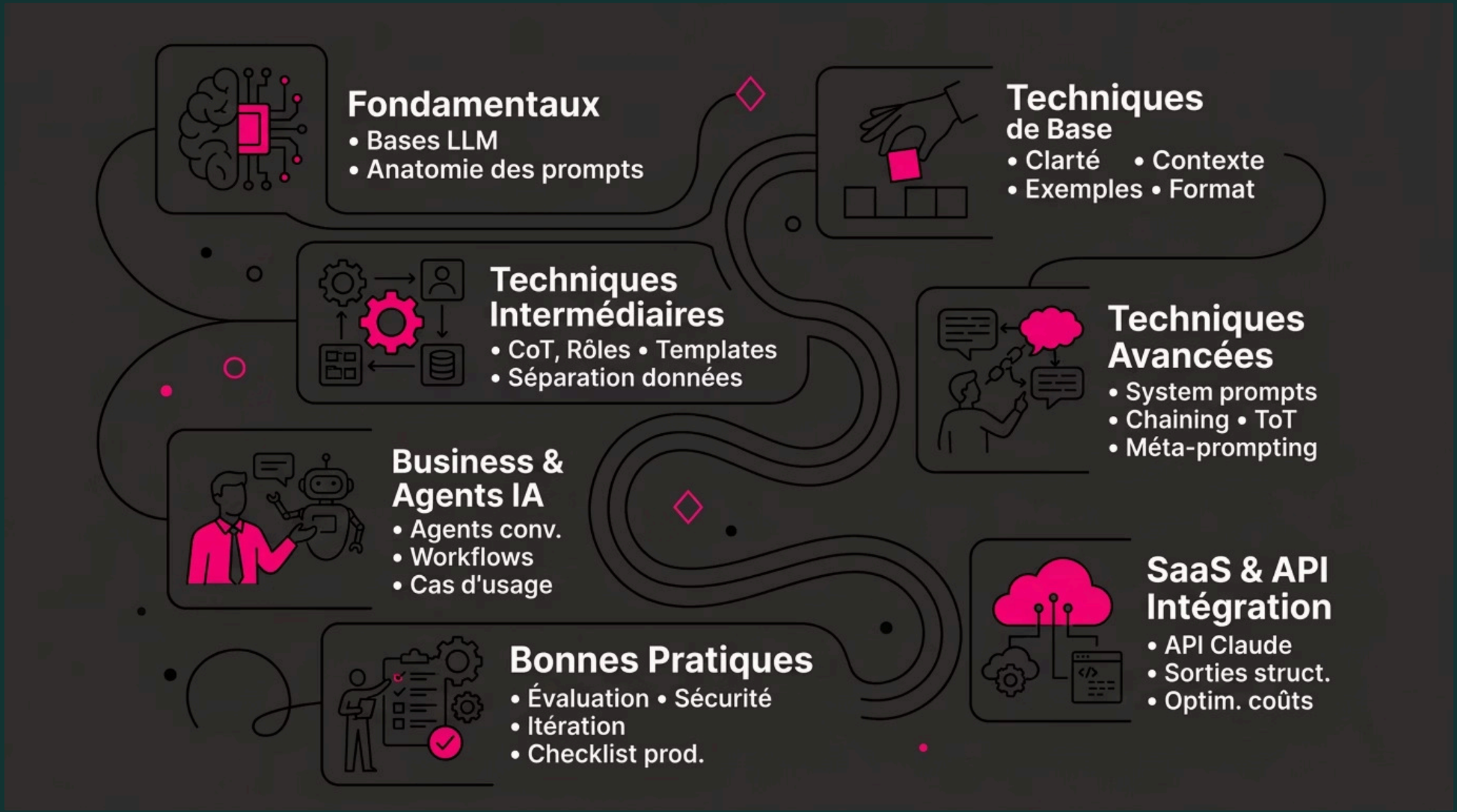
Ce que vous avez appris

En parcourant ces 7 modules, vous avez acquis un ensemble complet de compétences en prompt engineering, du niveau débutant au niveau expert. Voici un aperçu de l'ampleur du parcours que vous venez de compléter :

7
Modules
Du débutant à l'expert, un parcours progressif et complet

25+
Techniques
Directement actionnables dans vos projets dès aujourd'hui

∞
Applications
Business, SaaS, Agents IA — les possibilités sont illimitées



Ressources pour Aller Plus Loin

Votre parcours de prompt engineering ne fait que commencer. Voici les meilleures ressources pour continuer à progresser, pratiquer et rester à jour dans ce domaine en constante évolution. L'apprentissage continu est la clé — les modèles évoluent, les techniques se raffinement, et de nouvelles possibilités émergent chaque mois.

Documentation Anthropic

docs.anthropic.com — Guide officiel du prompt engineering, API reference, cookbook. La ressource la plus complète et la plus à jour pour maîtriser Claude et ses capacités. Consultez-la régulièrement pour rester informé des nouvelles fonctionnalités.

Cours Anthropic (GitHub)

github.com/anthropics/courses — Tutoriels interactifs gratuits, exercices pratiques. Le meilleur moyen d'apprendre en faisant, avec des notebooks Jupyter que vous pouvez exécuter directement.

Anthropic Prompt Library

docs.anthropic.com/en/prompt-library — Bibliothèque de prompts prêts à l'emploi. Des templates testés et optimisés pour une variété de cas d'usage. Idéal pour s'inspirer et démarrer rapidement.

Communauté & Pratique

La meilleure façon d'apprendre : **construire vos propres agents et itérer**. Chaque projet est un exercice de prompt engineering. Rejoignez des communautés, partagez vos découvertes, et n'ayez pas peur d'expérimenter.

Le meilleur prompt

est celui que vous allez écrire maintenant.

Prompt Engineering — Cours Complet

Techniques · Business & Agents IA · Intégration SaaS & API

Vous avez maintenant toutes les clés en main. Les fondamentaux pour comprendre, les techniques pour exceller, les méthodes pour industrialiser, et les bonnes pratiques pour sécuriser. Il ne reste plus qu'une chose : **passer à l'action**. Ouvrez votre IDE, lancez votre premier appel API, construisez votre premier agent. Chaque prompt que vous écrirez vous rendra meilleur. Le voyage commence ici. 🚀